SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)
(Established under section 3 of the UGC Act, 1956)
Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

PROJECT REPORT

ON

FOOD DELIVERY TIME PREDICTION

Using Three Kaggle Datasets

An End-to-End Predictive Analytics / KDD Project

Submitted By

Aaditi Gaikwad

PRN

23070521052

Submitted To

Dr. Deepak Asudani

Course

Predictive Analytics

13 October 2026

1. Introduction

Problem statement

Food orders take different amounts of time to arrive. A fixed estimate does not reflect differences between orders, conditions or delivery settings. This project uses historical records to predict a recorded delivery duration in minutes. Because the output is a continuous number, this is a supervised regression problem. Estimates may support planning and communication, but the study does not claim to reduce actual delays.

Objectives

Apply the Knowledge Discovery in Databases (KDD) framework to three Kaggle datasets, from source selection and cleaning through model comparison, evaluation and interpretation. Each dataset must contribute an executed experiment with its own held-out test and saved model. Compare the selected model with a simple median baseline, and explain errors in minutes rather than claim classification accuracy.

Why three separate experiments?

Dataset A is Den_Kuznetz's Food Delivery Time Prediction. Dataset B is Rajat Kumar's Food Delivery Time. Dataset C is Ahsan Raza's NYC Restaurants Data - Food Ordering and Delivery (FoodHub orders). They have different inputs and no verified common order key. They remain separate rather than being joined or concatenated. [1-3]

C explicitly separates delivery time from food preparation time. Its model predicts only the delivery stage. The exact start/end events for A and B are less clearly documented. Although all outcomes use minutes, their errors are separate benchmarks and cannot establish a universal ranking across sources.

Scope and intended use

A uses supplied distance and context. B assumes assigned-courier details are available. C uses order cost, cuisine and weekday/weekend. Actual preparation durations, post-order ratings and identifiers are excluded from the relevant predictor sets. No system is deployed, and no future-period performance is established because reliable order dates are absent.

Chapter	Content	Page
1. Introduction	Problem, objectives and scope	2
2. Literature Review	Research and methodology	3
3. Workflow / Predictive Modelling Process	Sources, cleaning, splitting and training	4-5
4. Results and Discussion	Scores, examples and interpretation	6-7
5. Conclusion	Outcomes, limitations and future work	8
6. References	All three Kaggle links and methods	9

2. Literature Review

Delivery-time prediction research

Garg et al. (2025) compare regression approaches for food delivery-time prediction in Indian cities. Their public abstract motivates evaluating linear and ensemble methods. This project uses its own data partitions and measured scores, rather than treating reported results from another study as directly comparable. [4]

Wen et al. (2023) survey service route and time prediction in instant delivery. This distinction clarifies the scope: the present project estimates duration, but does not optimize routes or assign delivery staff. A time estimate can be useful without solving route planning. [5]

Validation and preprocessing

The scikit-learn pipeline documentation explains how transformations can be included in fitting. Here, imputation and encoding learn only from each training fold. Cross-validation selects a model inside the development data, while a separate holdout is reserved for final scoring. [6,7]

Entity-group splitting prevents the same grouping entity from appearing in both training and evaluation. B separates couriers; C separates restaurants. A lacks entity identifiers and therefore uses a random order split. These designs answer different questions about generalization. C may still share customers across restaurants, so its split does not demonstrate performance on unseen customers. [6]

Interpreting a trained model

Permutation importance measures how much performance deteriorates when one input is shuffled. A larger error increase suggests that the fitted model depends on that input. It is an association, not a causal effect, and small differences can reflect sampling noise. Importance is calculated after selection and does not trigger further test-driven tuning. [8]

Source	Contribution to this implementation
Garg et al. [4]	Comparison of linear and ensemble regression.
Wen et al. [5]	Clear distinction between duration estimation and routing.
Cross-validation / pipelines [6,7]	Fold-local preprocessing and entity-separated evaluation.
Permutation importance [8]	Model interpretation with explicit causal limits.

The literature discussion is based on public abstracts and official documentation. The contribution is a reproducible educational workflow applied to three different sources, rather than a new prediction algorithm.

3. Workflow / Predictive Modelling Process

3.1 Dataset selection and data quality

Property	A	B	C
Owner	Den_Kuznetz	Rajat Kumar	Ahsan Raza
Raw rows	1,000	45,593	1,898
Raw columns	9	11	9
Usable rows	1,000	45,334	1,898
Target	Delivery_Time_min	Time_taken(min)	delivery_time
Inputs	Distance, experience, weather, traffic, time, vehicle	Derived distance, age, order type, vehicle	Order cost, cuisine, weekday/weekend
Grouping	No entity IDs	Courier ID	Restaurant name

A: missing information and prediction timing

A has 120 missing predictor cells: 30 each in courier experience, weather, traffic and time of day. Targets are complete and positive, and IDs do not repeat. The model excludes Order_ID and recorded Preparation_Time_min, because actual preparation time may only become available after the prediction is requested. [1]

B: repeated IDs and coordinate-derived distance

All 259 rows bearing repeated order IDs are excluded rather than choosing an arbitrary conflicting record. This leaves 45,334 rows. Haversine distance gives straight-line kilometres, not road travel distance. Coordinates must be nonzero and in valid ranges; derived distances over 100 km become missing. Age outside 18-80 also becomes missing. These fixed input-only rules mark 4,043 distances and 37 ages unavailable. Rows stay in the experiment for training-fold imputation. Ratings are excluded because their measurement timing is unclear. [2]

C: FoodHub order data

C has 1,898 unique orders across 178 restaurants. Selected predictors have no missing values and all targets are positive. Order cost, cuisine and weekday/weekend are model inputs. The model excludes identifiers, restaurant name as a predictor, rating and actual food preparation time. Restaurant name only defines split groups. Its target is the delivery stage, excluding preparation. [3]

Raw files remain unchanged. The notebook records checksums, exclusions, split membership and transformed missingness. Collection provenance is limited, so no source is relabelled as independently verified operational data.

3. Workflow (continued)

3.2 KDD stages, splits and modelling

Stage	Implementation
Understand	Audit schemas and targets; define inputs available at prediction time.
Prepare	Strip text whitespace; numeric median imputation plus missing indicators and scaling; Unknown categories and one-hot encoding.
Train	Compare four fixed model families using five validation folds within development data.
Evaluate	Choose lowest validation MAE, refit on development rows, and score the untouched holdout against a median baseline.
Interpret / reuse	Inspect importance and error examples; save and reload one complete pipeline per source.

Dataset	Development	Final test	Split unit
A	800 orders	200 orders	Random orders
B	36,397 orders / 1,056 couriers	8,937 orders / 264 couriers	Courier
C	1,600 orders / 142 restaurants	298 orders / 36 restaurants	Restaurant

Seed 42 fixes all random splits. B and C reserve 20% of entities, which need not produce exactly 20% of rows. GroupKFold creates five development-only folds. Assertions check zero group overlap in every fold and holdout. A uses five shuffled folds. No candidate sees final-test information during selection.

Candidate	Settings and purpose
Median baseline	Always predicts the training median.
Ridge	Alpha 10; additive weighted combination of inputs.
Random forest	100 trees, minimum leaf size 5, seed 42; averages trees.
Histogram gradient boosting	200 iterations, learning rate 0.05, 15 leaves, early stopping disabled, seed 42; learns successive corrections.

All candidates use identical partitions within a source. Learned preprocessing stays inside each fit. Predictions are clipped at zero without rounding before scoring. The exact versions and code appear in the executed notebook. The project does not transfer a trained model between sources.

4. Results and Discussion

4.1 Model selection and final evaluation

Candidate	A: CV MAE	B: CV MAE	C: CV MAE
Median baseline	17.916	7.559	4.159
Ridge	9.376	6.753	3.616
Random forest	10.098	6.738	3.640
Gradient boosting	10.490	6.502	3.615

Ridge wins A. Gradient boosting wins B and C. On C, boosting (3.6146 minutes) and Ridge (3.6160) are almost tied; the selection follows the predetermined minimum-MAE rule, but this tiny difference is not evidence of significant superiority. Fold variability is recorded in the result files.

Dataset / model	Test MAE	RMSE	R-squared
A: Baseline	17.47	21.18	-0.001
A: Ridge	8.57	11.14	0.723
B: Baseline	7.61	9.52	-0.022
B: Gradient boosting	6.48	8.04	0.271
C: Baseline	4.28	5.08	-0.031
C: Gradient boosting	3.64	4.36	0.242

MAE is the average size of a prediction error in minutes. RMSE gives larger misses more weight. R-squared describes explained variation, not classification accuracy. The selected models reduce MAE relative to their own baselines by **50.9% for A, 14.9% for B and 14.8% for C**.

A's error is 8.57 minutes, B's is 6.48 and C's is 3.64. These values are not a controlled comparison of algorithms: targets, duration ranges, features and split rules differ. C measures delivery excluding preparation. Adding C does not improve or retrain A into a combined model.

Within-ten-minute rates are 66.5% for A, 78.6% for B and 100% for C. C's baseline also reaches 100%, so this threshold does not distinguish useful performance there. It is not perfect prediction or a service guarantee.

4. Results (continued)

4.2 Examples, errors and model interpretation

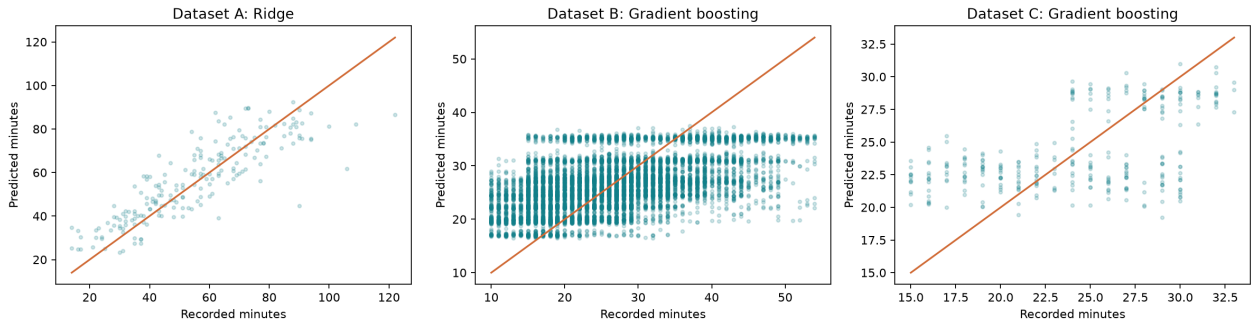


Figure 1. Predicted versus recorded test durations for the three selected models. The diagonal indicates a perfect prediction. Each panel uses a different source and test population.

Held-out example	Predicted	Recorded
A	37.62 min	32.00 min
B	27.08 min	36.00 min
C	22.25 min	20.00 min

These examples are the first held-out rows, not live orders. A predicts 37.62 instead of 32 minutes; B predicts 27.08 instead of 36; C predicts 22.25 instead of 20. The saved pipelines reproduce the same outputs when reloaded.

Dataset	Most influential inputs	Interpretation
A	Distance	Largest error increase when shuffled.
B	Distance and age	Nearly equal importance; no reliable causal ranking.
C	Weekday/weekend	Stronger dependence than cost or cuisine.

Permutation importance uses five shuffles per feature. C's weekday/weekend increases MAE by about 1.37 minutes when shuffled. Cost and cuisine have small negative estimates, meaning their shuffling did not harm performance in this sample. Negative values remain in the outputs instead of being set to zero. The analysis does not imply that a feature causes delivery delays. [8]

Training MAEs are 9.20, 6.43 and 3.25 minutes for A, B and C respectively. C's training error is lower than its 3.64-minute test error, which reinforces the need for a reserved test. No test-based tuning follows this evaluation.

5. Conclusion

Outcome

The project now uses three distinct Kaggle datasets in three executed KDD experiments. A selects Ridge, while B and C select gradient boosting. Each selected model improves on its own median baseline. The project includes source data, cleaning decisions, cross-validation, final tests, interpretation and reusable model pipelines.

What each source contributes

A demonstrates contextual features such as weather and traffic. B introduces coordinate-derived distance, repeated-ID handling and unseen-courier evaluation. C introduces order-level restaurant data and unseen-restaurant evaluation. This is substantive use of a third source, not merely another reference in the report.

Limitations

The sources do not provide reliable order timestamps, so future-period performance is untested. Target definitions differ, especially C's delivery-stage-only target. A lacks entity IDs; B's grouped test covers new couriers but not necessarily new restaurants or cities; C may share customers across restaurant partitions. Source provenance and availability of contextual features are not fully established.

B's distance screen is an explicit assumption and straight-line distance is not a road-route estimate. Age associations are descriptive and should not support employment decisions. C has no distance or traffic inputs and a narrow target range; its 100% ten-minute tolerance rate is also achieved by the baseline. None of the models produces calibrated arrival-time intervals.

Future work

Collect timestamped data with clearly defined duration endpoints and verified prediction-time inputs. Add road distances and genuine order-time preparation estimates. Test later periods and new locations, assess errors across relevant groups and provide uncertainty ranges. Operational use would also require monitoring and validation beyond these educational benchmarks.

Delivered project

The submission contains an executed Jupyter notebook covering all three datasets, the three raw CSV files, metadata and attribution, README, dependency file, shared feature preparation, inference code, three fitted models, results, this report and a ten-slide animated presentation. Speaker notes provide additional explanation without crowding the slides.

6. References

[1] Den_Kuznetz. Food Delivery Time Prediction. Kaggle, version 1. Apache 2.0 per metadata.
https://www.kaggle.com/datasets/denkuznetz/food-delivery-time-prediction

[2] Rajat Kumar. Food Delivery Time. Kaggle, version 1. ODbL / DbCL per metadata.
https://www.kaggle.com/datasets/rajatkumar30/food-delivery-time

[3] Ahsan Raza. NYC Restaurants Data - Food Ordering and Delivery. Kaggle, version 1. CC0 per metadata.
https://www.kaggle.com/datasets/ahsan81/food-ordering-and-delivery-app-dataset

[4] Garg et al. (2025). Food Delivery Time Prediction in Indian Cities Using Machine Learning Models.
https://arxiv.org/abs/2503.15177

[5] Wen et al. (2023). A Survey on Service Route and Time Prediction in Instant Delivery.
https://arxiv.org/abs/2309.01194

[6] scikit-learn. Cross-validation and grouped observations.
https://scikit-learn.org/stable/modules/cross_validation.html

[7] scikit-learn. Pipelines and composite estimators.
https://scikit-learn.org/stable/modules/compose.html

[8] scikit-learn. Permutation feature importance.
https://scikit-learn.org/stable/modules/permutation_importance.html

Dataset attribution and licenses

Contains information from Food Delivery Time by Rajat Kumar, made available under the Open Database License (ODbL), with contents under DbCL as listed in Kaggle metadata. Original raw files, metadata and source notices are included. A and C retain their own distinct license notices.

ODbL: https://opendatacommons.org/licenses/odbl/1-0/
DbCL: https://opendatacommons.org/licenses/dbcl/1-0/
CC0: https://creativecommons.org/publicdomain/zero/1.0/